TIP The HTTP client plugin in JetBrains Rider makes it easy to craft HTTP requests from inside your API, and even discovers all the endpoints in your application automatically, making them easier to test. You can read more about it at https://www.jetbrains.com/help/rider/Http_client_in__product__code_editor.xhtml.

As a final note before we move on, it's worth mentioning the behavior you get when you call a method with the *wrong* HTTP verb. If you define an API like the one in listing 5.1

```
app.MapGet("/person/{name}", (string name) =>
  people.Where(p => p.FirstName.StartsWith(name)));
```

and call it by using a `POST` request to `/person/Al` instead of a `GET` request, the handler won't run, and the response you get will have status code `405 Method Not Allowed`.

TIP You should never see this response when you're calling the API correctly, so if you receive a `405` response, make sure to check that you're using the right HTTP verb and the right path. Often when I see a `405`, I've used the correct verb but made a typo in the URL!

In all the examples in this book so far, you provide a lambda function as the handler for an endpoint. But in section 5.2.3, you'll see that there are many ways to define the handler.

5.2.3 Defining route handlers with functions

For basic examples, using a lambda function as the handler for an endpoint is often the simplest approach, but you can take many approaches, as shown in the following listing. This listing also demonstrates creating a simple